AI Coding-Agent Harnesses: What Changed, What to Mandate

Executive brief · harnesswatch project · October 2026

Bottom line. AI coding agents are now judged by their *harness*: the runtime that holds credentials, runs shell commands, loads repository configuration, starts MCP servers and runs unattended in CI. Much of the practical risk now sits in the harness, not the model. Across 323 verified harness incidents and CVEs disclosed in 2025–2026 (99 in 2025, 224 in 2026), the most common failure class was a weak or bypassable permission model (23% of incidents). The most common root cause was a validation, parsing or authorization flaw in the very check that gates what the agent may do (32%). Advisories are typically published once a fix exists (135 of the 148 incidents with a known fix date), so the exposure that remains is mostly *configuration and defaults*. Patching alone does not remove it.

What changed in 2025–2026

- **Repositories became code.** Hook settings, MCP server lists and instruction files committed to a repository run or steer the agent for everyone who clones it. We sampled public files by type. 51.8% of the sampled agent config files declared MCP servers and 28.3% launched them unpinned from a registry. Among sampled agent CI workflows, 63.3% granted write-scoped tokens and 28.2% ran the agent on untrusted triggers.
- **Closed clients can be checked, and mostly match their documentation.** We tested 28 of 117 documented security claims on the shipped clients, with no vendor account: 22 matched and 1 did not. The rest need interactive sessions or vendor logins that we did not have. "Documented" is not "safe". For example, one widely used client documents that in headless mode it runs a repository's hooks and MCP servers without a trust prompt. Our probe confirmed it.
- **Supply chain is thin.** Of 23 harness packages, 17 offer a `curl | bash` installer that verifies no signature. Only 10 of 18 checkable packages publish registry provenance. The median package pulls 97 transitive dependencies.
- **Secrets leak into local state.** When an agent reads a secret, 6 of 7 harnesses we ran kept a copy in transcripts or state files. In 3, that copy was readable by other users on the machine.

Top five risks in business terms

1. **A cloned repository runs code on the developer's machine or in CI.** Repository hooks, MCP servers and env overrides run with the developer's credentials.
2. **Unattended CI agents with write tokens act on attacker-written text.** This is the "pwn request" pattern: a poisoned issue or PR steers an agent that holds repository secrets.
3. **Auto-approve modes turn any injection into action.** In our sandboxed benchmark, auto-approve mode completed real work in 84 of 135 runs, against 23 of 135 in default mode. Productivity pressure pushes teams towards auto-approve.
4. **Unvetted extensions.** MCP servers, plugins and skills are installed unpinned from public registries.
5. **Secret sprawl and telemetry.** Copies of secrets end up in agent logs, and data leaves through vendor telemetry that teams do not inventory.

What to mandate before rollout

Control (from the governance checklist)	Verify with
Managed settings that lock bypass/auto-approve off and pin the permission mode	`harnesswatch audit config` (HG-PERM-*)
Untrusted repositories cannot run hooks or start MCP servers; trust prompts are on	`harnesswatch audit config` (HG-HOOK-001, HG-MCP-005)
MCP servers pinned, from an allowlist, authenticated, no inline secrets	`harnesswatch audit mcp` (HG-MCP-001…006)

Control (from the governance checklist)	Verify with
Headless agents never run on untrusted triggers with write tokens	`harnesswatch audit ci` (HG-CI-001/002)
Agent state directories private; secrets kept out of transcripts	`harnesswatch audit secrets` (HG-SEC-001/002)
Signed installers, pinned versions, controlled auto-update; SBOM on file	`harnesswatch audit supply`
Vendor claims verified on your own build before approval	`harnesswatch observe`

Decision table

Harness class	Acceptable when
Local CLI/IDE agent, interactive, default permissions	Managed settings enforced; repo-trust prompts on; MCP allowlist; state dirs private
Same, auto-approve / bypass mode	Only inside a disposable sandbox (container/VM) with no production credentials and egress allowlisting
Headless agent in CI	Only on trusted triggers, least-privilege token, no secrets in the job, output reviewed by a human
Closed-source harness	As above, **plus** a black-box check of its documented controls on your version (`harnesswatch observe`)
Agent SDK / managed runtime	Treat as application code: threat model, code review, the same controls built in

Cost of inaction

323 verified incidents in two years, across every major vendor. Most were patched quickly, but each exploited a default or a trust boundary that an organisation could have locked down in advance. The full evidence is in the paper and its evidence ledger. The governance checklist maps each control to NIST AI RMF, ISO/IEC 42001, OWASP, MITRE ATLAS and the MAS AI Risk Management Guidelines.

Numbers come from the project's results files (experiments/results/.json), generated by `make experiments`. Limits of the study, such as local small models in the benchmark and headless-only probes, are in paper §Limitations.*